SOLVING PROBLEMS BY SEARCHING

KECERDASAN BUATAN

Dosen Pengajar:

I Putu Edy Suardiyana Putra, Ph.D.

Nama Anggota:

Muhammad Abdan Syakuron	2207421021
Dhea Salsabela	2207421029
Syifa Chandra Tiffani Sumardi	2207421030

TMJ 8A

PROGRAM STUDI TEKNIK MULTIMEDIA DAN JARINGAN

JURUSAN TEKNIK INFORMATIKA DAN KOMPUTER

POLITEKNIK NEGERI JAKARTA

TAHUN AJARAN 2025/2026

Exercise 1

Pengorganisasian file digital merupakan masalah yang sering muncul dalam sistem manajemen dokumen. Misalnya, seorang pengguna ingin menyusun 6 file ke dalam beberapa folder digital. Setiap file memiliki tipe tertentu (misalnya PDF, DOCX, atau JPG), dan terdapat aturan bahwa tidak boleh ada lebih dari dua file dengan tipe yang sama dalam satu folder.

Meskipun masalah ini dapat diselesaikan dengan algoritma khusus, masalah ini juga dapat digunakan sebagai contoh masalah pencarian (search problem) dalam kecerdasan buatan.

1. **States:** Setiap state merepresentasikan kondisi susunan file yang sudah ditempatkan ke dalam folder. State harus menyimpan informasi:

- File mana saja yang sudah ditempatkan.
- Folder mana yang berisi file tersebut.
- Jumlah file dari tiap tipe dalam setiap folder.

Contoh representasi state:

State = (s1, s2, s3, s4, s5, s6)

si menunjukkan folder tempat file Fi ditempatkan

Fi = nama file

si = elemen state

si ∈ {Folder1, Folder2, Folder3, 0}

0 = file belum ditempatkan.

Dengan aturan tidak boleh ada lebih dari 2 file dengan tipe yang sama dalam satu folder.

Contoh:

(Folder1, Folder2, Folder1, 0, 0, 0)

Artinya:

- F1 → Folder1
- F2 → Folder2
- F3 → Folder1

- F4, F5, F6 belum ditempatkan

2. **Initial State:** Semua file belum ditempatkan ke dalam folder.

   S0 = (0, 0, 0, 0, 0, 0)

   Artinya:

   Folder1 = {}

   Folder2 = {}

   Folder3 = {}

   Remaining = {F1, F2, F3, F4, F5, F6}

3. **Action**: Aksi yang dapat dilakukan adalah menempatkan satu file yang belum ditempatkan ke dalam salah satu folder yang tersedia, selama aturan tidak dilanggar (maksimal dua file dengan tipe yang sama dalam satu folder).

   Misalnya, F1 dimasukkan ke Folder1

   S1 = (Folder1, 0, 0, 0, 0, 0)

   Atau

   Folder1 = {F1}

   Folder2 = {}

   Folder3 = {}

   Remaining = {F2, F3, F4, F5, F6}

4. **Transition Model:** Jika sebuah file ditempatkan ke suatu folder, maka:

- File tersebut dianggap sudah ditempatkan.
- Folder tujuan akan bertambah satu file dengan tipe tertentu.
- Contoh:

  Jika aksi = menempatkan File1 (PDF) ke Folder1,

  maka state baru menjadi:

  Folder1 = {File1(PDF)}

  State baru juga harus tetap memenuhi constraint bahwa tidak boleh ada lebih dari dua file dengan tipe yang sama dalam satu folder.

5. **Goal Test:** Tujuan tercapai jika:

- Semua 6 file sudah ditempatkan ke dalam folder, dan
- Tidak ada folder yang memiliki lebih dari dua file dengan tipe yang sama.

Contoh:

Sgoal = (Folder1, Folder2, Folder1, Folder3, Folder2, Folder3)

Artinya:

Folder1 = {F1, F3}

Folder2 = {F2, F5}

Folder3 = {F4, F6}

What is the branching factor at the initial state?

Branching Factor

Pada initial state terdapat k folder yang dapat dipilih untuk menempatkan file pertama. Oleh karena itu branching factor pada state awal adalah:

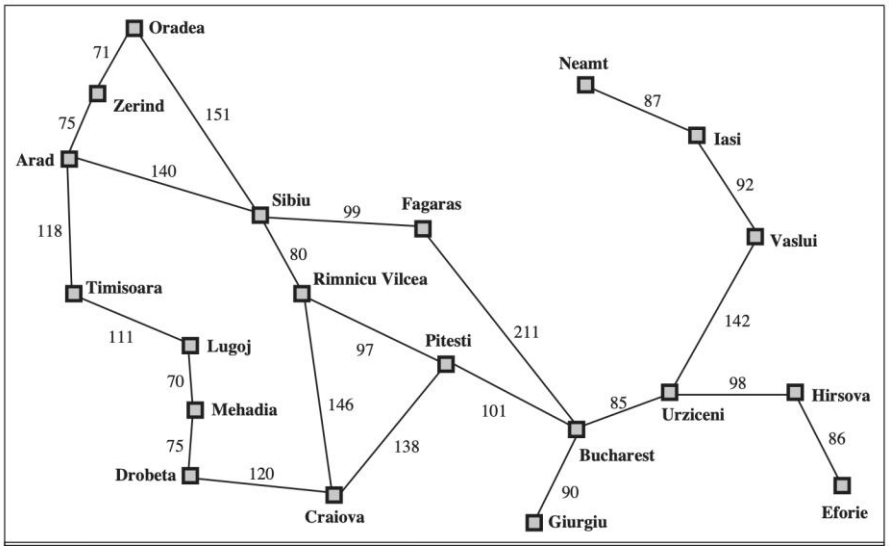
$$k$$

Sebagai contoh, jika terdapat 3 folder, maka branching factor = 3.

6. **Path Cost:** Dalam masalah ini, path cost biasanya tidak terlalu penting, karena tujuan utamanya hanya mencari susunan file yang valid sesuai aturan. Setiap langkah penempatan file dapat dianggap memiliki biaya yang sama, sehingga yang dicari hanyalah solusi yang memenuhi constraint, bukan solusi dengan biaya minimum.

Dengan demikian, masalah pengorganisasian file dapat diformulasikan sebagai search problem dalam kecerdasan buatan, di mana algoritma pencarian dapat digunakan untuk menemukan susunan file yang memenuhi semua constraint yang diberikan.

EXERCISE 2



Tugas

Dengan menggunakan metode Tree Search, temukan empat jalur berikut antara kota-kota yang diberikan. Untuk setiap masalah:

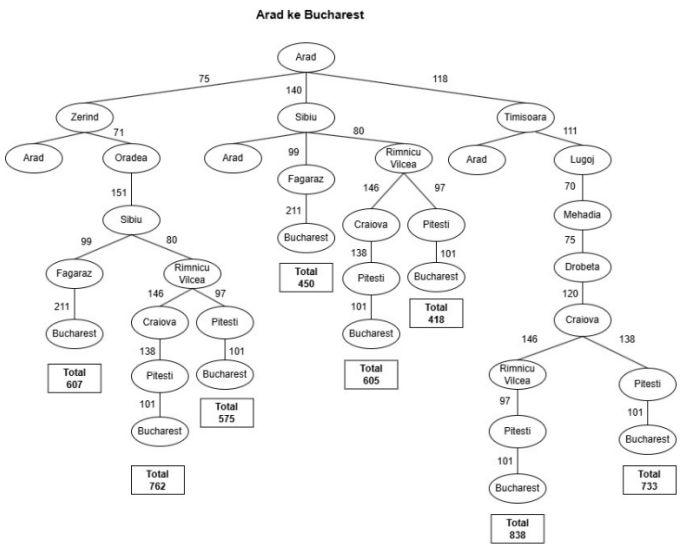
- Gambarkan atau tuliskan ekspansi pohon pencarian (search tree expansion).
- Tunjukkan urutan kota yang dikunjungi.
- Hitung total biaya jalur (jarak).

Untuk setiap permasalahan, tampilkan:

- Urutan ekspansi pohon pencarian (search tree expansion order)
- Jalur akhir yang ditemukan (final path)
- Total jarak (total distance)
- Penjelasan singkat mengenai langkah-langkah pencarian (search steps)

Permasalahan

- Temukan jalur dari Arad ke Bucharest.



Urutan node yang dikembangkan selama proses pencarian:

Arad → Zerind → Oradea → Sibiu → Fagaras → Bucharest

Arad → Sibiu → Fagaras → Bucharest

Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest

Arad → Timisoara → Lugoj → Mehadia → Drobeta → Craiova → Pitesti → Bucharest

Jalur terbaik yang ditemukan berdasarkan jalur dengan jarak terkecil pada tree:

Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest

Perhitungan jarak:

Arad → Sibiu = 140

Sibiu → Rimnicu Vilcea = 80

Rimnicu Vilcea → Pitesti = 97

Pitesti → Bucharest = 101

Total jarak untuk nodes dengan jarak terdekat:

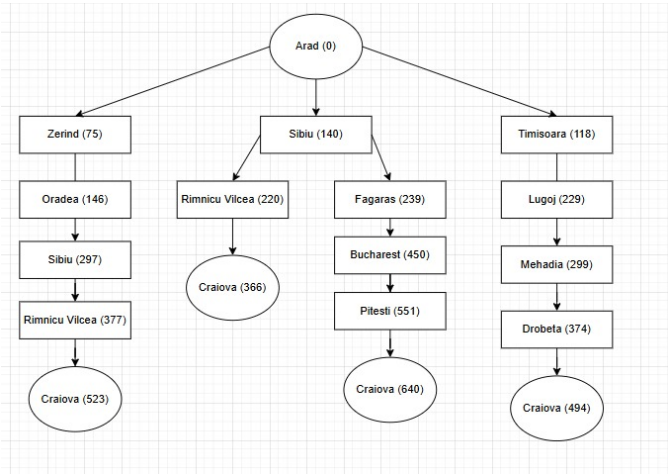
140 + 80 + 97 + 101 = 418 km

Proses pencarian dimulai dari kota Arad sebagai node awal. Dari Arad terdapat tiga kemungkinan jalur yaitu menuju Zerind, Sibiu, dan Timisoara. Setiap kota yang ditemukan kemudian dikembangkan menjadi node baru dalam pohon pencarian. Dari cabang Zerind dan Timisoara ditemukan jalur menuju Bucharest namun dengan jarak yang lebih panjang. Cabang melalui Sibiu menghasilkan beberapa kemungkinan jalur.

Jalur Sibiu → Rimnicu Vilcea → Pitesti → Bucharest menghasilkan jarak paling pendek.

Setelah node Bucharest ditemukan, proses pencarian berhenti dan jalur tersebut dipilih sebagai solusi.

- Temukan jalur dari Arad ke Craiova.



Final Path: Arad → Sibiu → Rimnicu Vilcea → Craiova

Total Distance: Arad - Sibiu - Rimnicu Vilcea = 366

Penjelasan: Mulai dari Arad, ekspansi menghasilkan 3 tetangga: Zerind (75), Sibiu (140), Timisoara (118)

Ekspansi Zerind → menghasilkan Oradea (146). Bukan tujuan, lanjut.

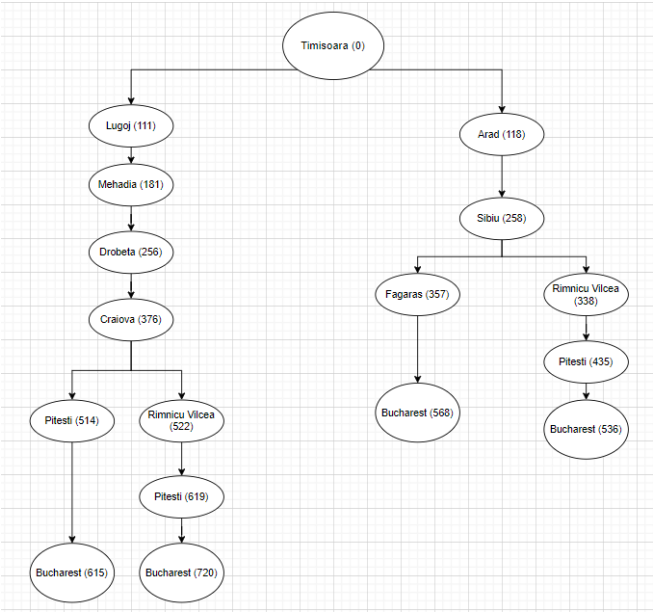
Ekspansi Sibiu → menghasilkan Rimnicu Vilcea (220) dan Fagaras (239). Bukan tujuan, lanjut.

Ekspansi Timisoara → menghasilkan Lugoj (229). Bukan tujuan, lanjut.

Ekspansi Oradea → menghasilkan Sibiu (297). Bukan tujuan, lanjut.

Ekspansi Rimnicu Vilcea → menghasilkan Craiova (366). Tujuan rute tercepat ditemukan

- Temukan jalur dari Timisoara ke Bucharest.



Urutan node yang diekspansi berdasarkan tree pada gambar:

Timisoara → Lugoj → Mehadia → Drobeta → Craiova → Arad → Sibiu → Fagaras → Rimnicu Vilcea → Pitesti → Bucharest

Jalur Akhir yang Ditemukan (Final Path)

Berdasarkan jalur dengan jarak terkecil pada tree:

Timisoara → Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest

Total Distance tertulis pada gambar, untuk nodes dengan jarak terdekat:

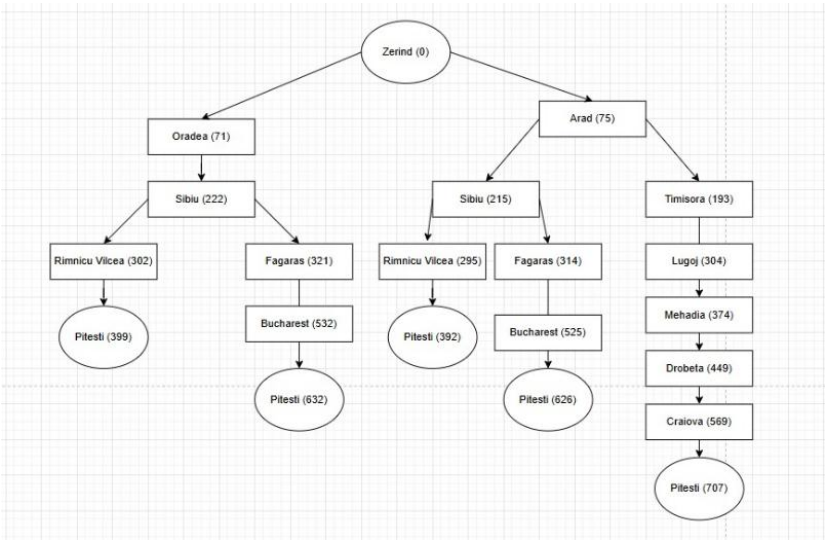
- Timisoara → Arad = 118
- Arad → Sibiu = 140
- Sibiu → Rimnicu Vilcea = 80
- Rimnicu Vilcea → Pitesti = 97
- Pitesti → Bucharest = 101

Total Distance:

$$118 + 140 + 80 + 97 + 101 = 536$$

Proses pencarian dimulai dari Timisoara (0). Dari node ini terdapat dua cabang yaitu menuju Lugoj dan Arad. Cabang kiri dieksplorasi melalui Lugoj → Mehadia → Drobeta → Craiova, kemudian menghasilkan node Pitesti dan Rimnicu Vilcea, tetapi jalur ini menghasilkan jarak yang lebih besar menuju Bucharest. Cabang kanan dari Arad menghasilkan Sibiu, yang kemudian bercabang menjadi Fagaras dan Rimnicu Vilcea. Jalur melalui Rimnicu Vilcea → Pitesti akhirnya mencapai Bucharest dengan total jarak 536 km, yang merupakan jalur terbaik pada pohon pencarian yang diberikan.

- Temukan jalur dari Zerind ke Pitesti.



Final Path: Zerind → Arad → Sibiu → Rimnicu Vilcea → Pitesti

Total Distance: Zerind→Arad→Sibiu→RimnicuVilcea→Pitesti = 392

Mulai dari Zerind, ekspansi menghasilkan 2 tetangga: Oradea (71) dan Arad (75)

Ekspansi Oradea → menghasilkan Sibiu (222). Bukan tujuan, lanjut.

Ekspansi Arad → menghasilkan Sibiu (215) dan Timisoara (193). Bukan tujuan, lanjut.

Ekspansi Sibiu (dari Oradea) → menghasilkan Rimnicu Vilcea (302) dan Fagaras (321). Bukan tujuan, lanjut.

Ekspansi Sibiu (dari Arad) → menghasilkan Rimnicu Vilcea (295) dan Fagaras (314). Bukan tujuan, lanjut.

Ekspansi Timisoara → menghasilkan Lugoj (304). Bukan tujuan, lanjut.

Ekspansi Rimnicu Vilcea (dari Oradea path) → menghasilkan Pitesti (399).

Ekspansi Rimnicu Vilcea (dari Arad path) → menghasilkan Pitesti (392). Tujuan rute tercepat ditemukan.